Date:

assignment -[1]-

Q.1 Algorithm Design and Analysis

1- Algorithm strategy

We use a single traversal of the array

- Set the first element as both min and max
- compare every remaining element with min and max
- if the element is smaller then min, update min,
- if the element is larger then max, update max,
- After checking all elements, min is the smallest and max is the largest. This algorithem scans the array only once

Ex-

Date:

2- pseudocode.

</> plain text

```
Algorithm findMinMax (A, n)

Input: An array A containing n integers

output: The smallest and largest elements

min ← A[0]
max ← A[0]

for i ← 1 to n-1 do

  if A[i] < min then
    min ← A[i]
  end if
if A[i] > max then

  max ← A[i]
  end if

end for

return min, max
```

Date:

[3] major primitive operations

The main operations are:

- Array access A[i]
- comparison A[i] < min
- comparison A[i] > max
- Assignment min ← A[i]
- Assignment max ← A[i]
- Incrementing the loop variable.

The algorithm performs 2 comparisons

for each of the n-1 remaining elements

, so approximately 2(n-1) comparisions.

[4] Best_case and worst_case Running Time

Even if the array is already sorted, we

still need to examine every element

Therefore:

$T_{best}(n) = \Theta(n)$

worst case:

we also examine every element

Therefore:

$T_{worst}(n) = \Theta(n)$

Date:

5) Asymptotic Notation

The running time is:
- Big-O : O(n)
- Big-Ω : Ω(n)
- Big-Θ : Θ(n)

Therefore the overall time complexity is : Θ(n).

6) space complexity

only a few extra variables are used:
- min
- max
- i

No additional array or data structure is required

Therefore:

space complexity = O(1)

2) Algorithm 1 : Linear search

7) Strategy

check each element from the beginning of the array until:
- x is found, or
- all elements have been checked.

Mohammed 71595

Date:

assignment -1-

8) pseudocode:

```
</> plain text
Algorithm Linear search (A, n, x)
for i ← 0 to n-1 do
if A[i] = x then
    return TRUE
end if
end for
return FALSE
```

9) Time complexity

Best case:
x is the first element,
$\Omega(1)$

Worst case:
x is the last element or doesn't exist. $O(n)$

10) Asymptotic complexity

- Best case: $\Omega(1)$
- worst case: $O(n)$
- General / tight bound: $\Theta(n)$.

Date:

Algorithm 2: Binary Search

Important: Binary search requires the array to be sorted

7 Strategy

Binary search repeatedly divides the search area into two halves,
- Check the middle element
- if (x) is smaller, search the left half
- if x it equals x, return TRUE.

8 Pseudocode

```
<!> plain text
low ← 0
high ← n-1
while low ≤ high do
  mid ← (low + high) / 2
  if A[mid] = x then
    return TRUE
  else if x < A[mid] then
      high ← mid - 1
  else
    low ← mid + 1
  end if
end while
return FALSE
```

Date:

(9) Time complexity

Best case:
The value x is found in the middle element in the first comparison $\Omega(1)$

Worst case:
The search continues until the search range becomes empty $O(\log n)$

(10) Asymptotic Complexity

- Best case: $\Omega(1)$
- worst case: $O(\log n)$
- overall: $O(\log n)$

(11) comparison

linear search works with sorted or unsorted arrays, but its worst-case time is $O(n)$.

Binary search is faster for large arrays because its worst-case time is $O(\log n)$ but the array must be sorted

Therefore, binary search is more efficient for large sorted arrays, while Linear Search is better for unsorted arrays.

Date:

12. Effect of input size and arrangement

For small arrays, both algorithms can work well. For large array Binary search is much faster if the array is sorted.

In Linear Search, the position of x affects the time : finding it at the beginning is faster, while finding it at the end or not finding it at all takes longer.

In Binary Search, the array must be sorted, and the search becomes smaller by half in each step.